# Practice Assignment 16.1: Deploying and Testing an Agentic API with FastAPI

This practice exercise helps you apply Week 16 concepts by deploying an **agent as a REST API** using **FastAPI** inside a Jupyter environment. You will simulate a real-world deployment workflow by defining an agent, exposing it through API endpoints, running the server locally, and testing it using HTTP requests. The focus is on understanding the end-to-end flow from **agent logic** to **API deployment** to **basic testing**.

- Installing and importing deployment libraries (FastAPI, uvicorn, nest_asyncio, requests)
- Defining an intelligent agent using LangChain and Hugging Face pipelines
- Implementing an agent response function that combines rule-based and LLM-based outputs
- Creating API endpoints for inference and status checks
- Running FastAPI inside Jupyter and testing with POST requests
- (Optional) Recording simple user feedback via an API endpoint

## Instructions

You will build and deploy a small agentic API in a notebook. Before you expose the agent via FastAPI, test the agent logic directly on a few example queries to confirm it behaves as expected. As you work, keep a short log of:

- The test queries you used
- The agent responses (rule-based vs LLM-based, if applicable)
- Any changes you made to improve clarity or stability
- The API test request and response payloads

### Task: Deploying and Testing an Agentic API with FastAPI

1. **Install and import the required libraries**, including FastAPI, *nest_asyncio*, uvicorn, and requests.
2. **Define an intelligent agent** using LangChain and Hugging Face pipelines.
3. **Implement the** `agent_response()` **function** to combine rule-based and LLM-based responses.
4. **Test the agent logic** on 4–5 queries before deploying it as an API.
5. **Create a FastAPI app** with the following endpoints:
    - `/` **(GET)** → Returns API status.
    - `/predict` **(POST)** → Takes a JSON query and returns the agent response.

6. **Run FastAPI inside Jupyter** using *nest_asyncio* and a *Thread* to simulate deployment.
7. **Test API responses** using POST requests within the same notebook.
8. **(Optional)** Add an endpoint `/feedback` to record user satisfaction ratings for agent responses.

## Expected Output

By the end of this practice, you should have:

- A working notebook that defines an agent and runs a FastAPI server locally inside Jupyter.
- A functional `/` status endpoint that confirms the API is running.
- A functional `/predict` endpoint that returns an agent response for a JSON query.
- 4–5 test queries showing the agent logic working before deployment.
- At least two successful POST request tests (request payload + response output) captured in the notebook.
- (Optional) A `/feedback` endpoint that stores simple user ratings.

## How to Approach the Project

Treat this as a deployment rehearsal, not a production build. Aim for a clean, testable workflow:

1. **Start with imports and environment:** Install and import libraries first, then confirm your notebook kernel recognises them.
2. **Validate agent logic early:** Implement `agent_response()` and test it directly with 4–5 queries before adding FastAPI.
3. **Build the API endpoints next:** Add `/` and `/predict` and keep responses simple and JSON-safe.
4. **Run the server inside Jupyter carefully:** Use *nest_asyncio* and a thread-based approach to avoid event-loop conflicts.
5. **Test like a user would:** Use requests to send POST payloads and verify you get consistent, expected responses.
6. **Add feedback only after the core works:** The optional `/feedback` endpoint is easiest once `/predict` is stable.

**Minimum completion standard (recommended):** Complete Tasks 1–7 and demonstrate at least two successful POST tests against `/predict`.

***Note:*** *Working through problems independently is key to building confidence and developing a lasting understanding. This is a self-study activity for your practice and does not count towards programme completion. No submission is required. However, we highly recommend completing the activity on Vocareum to enhance your overall learning experience.*